

Processamento Digital de Sinais

Soluções para Tutorial 15 - Modulação AM/FM e Transformada de Hilbert

https://github.com/fchirono/Aulas_PDS_Acustica

2 Tarefas

2.1 Geração dos Sinais Moduladores e Modulados

Tarefa 1

Veja o script `tutorial15_solucoes.py` para um exemplo de como esta Tarefa pode ser solucionada.

Tarefa 2

A função `gerador_seno_am` (no script `tutorial15_solucoes.py`) implementa diretamente a Equação (2) do enunciado, com a portadora $x_c(t) = \sin(2\pi f_c t)$ construída sobre um vetor de tempo de mesmo comprimento que `xmod`. O índice de modulação m é o valor de pico de `xmod`, e um aviso é emitido sempre que o índice de modulação $m > 1$ (sobremodulação).

Esta implementação foi adaptada da função `mosquito.utils.am_sine_generation` do pacote `MoSQITo`, disponível em <https://github.com/Eomys/MoSQITo>.

Tarefa 3

A função `gerador_seno_fm` (no script `tutorial15_solucoes.py`) implementa a Equação (5), integrando numericamente a frequência instantânea $f_{inst}(t) = f_c + k x_{mod}(t)$ por soma acumulada (`numpy.cumsum`) para obter a fase $\phi(t)$. O desvio máximo de frequência Δf e o índice de modulação β são calculados diretamente das definições da Seção 1.2 do enunciado.

Esta implementação foi adaptada da função `mosquito.utils.fm_sine_generation` do pacote `MoSQITo`, disponível em <https://github.com/Eomys/MoSQITo>.

Tarefa 4

Utilizando um sinal modulador de banda limitada (Tarefa 1), com portadora $f_c = 30$ Hz (para melhor visualização dos resultados neste documento) e sensibilidade em frequência $k = 50$ Hz, obtêm-se os sinais AM e FM das Figuras 1 e 2. Nota-se claramente a amplitude do sinal AM e a frequência instantânea do sinal FM variando de acordo com o sinal modulador, conforme esperado.

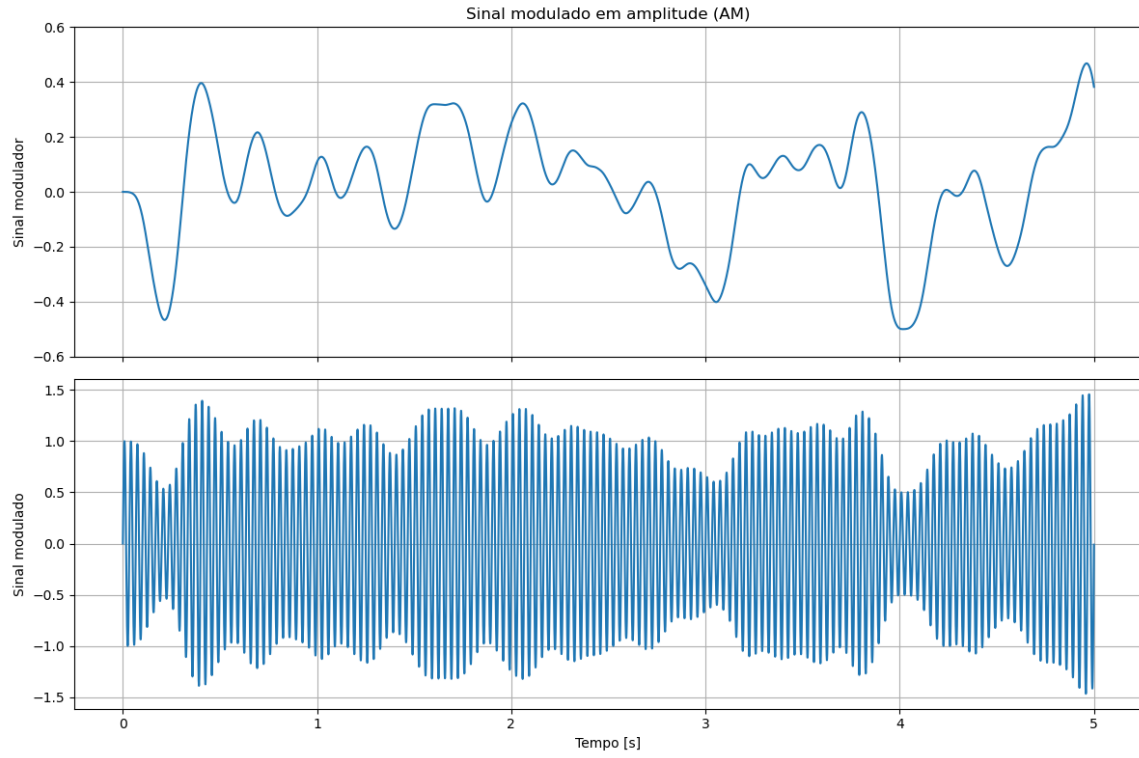


Figura 1: Sinal modulador de banda limitada (topo), sinal AM completo ($f_c = 30$ Hz).

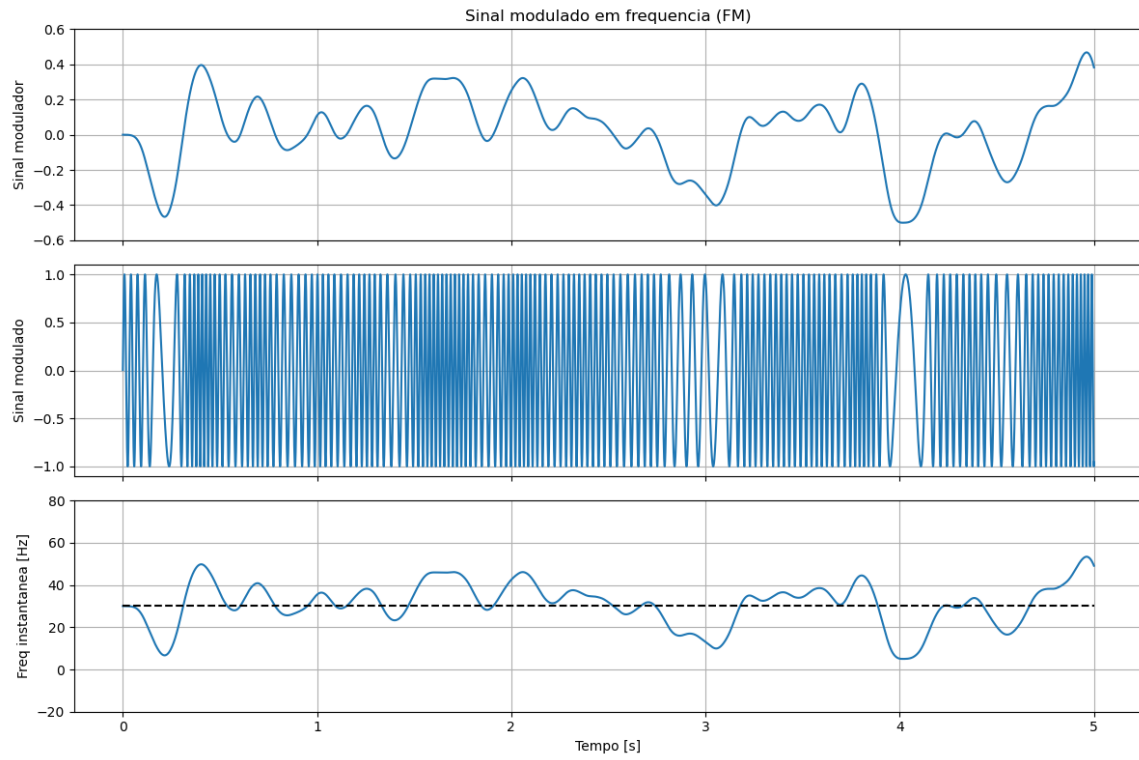


Figura 2: Sinal modulador de banda limitada (topo), sinal FM ($f_c = 30$ Hz), e frequência instantânea $f_{inst}(t)$.

2.2 Sinal Analítico via Transformada de Hilbert

Tarefa 1

Uma das principais propriedades do sinal analítico é possuir um espectro *unilateral* – isto é, contendo apenas frequências positivas (e a componente DC, quando existente). Partindo de $z(t) = x(t) + j\tilde{x}(t)$ e de $H(f) = -j \operatorname{sgn}(f)$ (resposta em frequência da Transformada de Hilbert, Equação 6), tem-se $\tilde{X}(f) = H(f)X(f)$, e portanto:

$$\begin{aligned} Z(f) &= X(f) + j H(f) X(f) \\ &= X(f) [1 + j (-j \operatorname{sgn}(f))] \\ &= X(f) [1 + \operatorname{sgn}(f)] \\ &= \begin{cases} 2 X(f), & f > 0 \\ X(f), & f = 0 \text{ ou } f = f_s/2 \\ 0, & f < 0 \end{cases} \end{aligned} \quad (1)$$

Ou seja, as componentes de frequência negativa de $X(\Omega)$ são canceladas, as de frequência positiva são dobradas em amplitude, e a componente DC e a frequência de Nyquist $f_s/2$ permanecem inalteradas.

Tarefa 2

Numericamente, o sinal analítico de uma sequência real $x[n]$ de comprimento par N pode ser calculado diretamente no domínio da frequência, explorando a propriedade (1): calcula-se a DFT $X[k]$ de $x[n]$, multiplica-se $X[k]$ por um vetor $h[k]$ que implementa a versão discreta de $1 + \operatorname{sgn}(\Omega)$,

$$h[k] = \begin{cases} 1, & k = 0 \text{ ou } k = N/2 \quad (\text{DC e Nyquist}) \\ 2, & k = 1, \dots, N/2 - 1 \quad (\text{frequências positivas}) \\ 0, & k = N/2 + 1, \dots, N - 1 \quad (\text{frequências negativas}) \end{cases}, \quad (2)$$

e retorna-se ao domínio do tempo pela IDFT: $z[n] = \text{IDFT}\{X[k] h[k]\}$. As componentes DC e Nyquist *não* são dobradas, pois são as únicas frequências que não possuem uma componente “espelhada” (negativa) distinta a ser descartada.

```
def calc_sinal_analitico(x):  
    """  
    Retorna o sinal analitico z = x + j*Hilbert(x), calculado no dominio  
    da frequencia a partir da propriedade de espectro unilateral.  
    """  
    N = x.shape[0]  
    Xf = np.fft.fft(x)  
  
    # vetor 'h[k]' para multiplicar o espectro de 'x'  
    h = np.zeros(N)  
  
    # frequencia zero (DC) e Nyquist sao multiplicadas por 1  
    h[0] = h[N // 2] = 1  
  
    # frequencias positivas sao multiplicadas por 2  
    h[1 : N // 2] = 2  
  
    # frequencias negativas permanecem multiplicadas por 0  
    return np.fft.ifft(Xf * h)
```

Nota importante: o resultado da IDFT é, em geral, complexo – a parte real reproduz exatamente $x[n]$ (a menos de erro numérico de ponto flutuante), e a parte imaginária é a Transformada de Hilbert $\tilde{x}[n]$.

A Figura 3 compara o sinal analítico de um tom puro $x[n] = \cos(2\pi f_0 n / f_s)$, calculado tanto por `calc_sinal_analitico` quanto pela função de referência `scipy.signal.hilbert`. As partes real e imaginária das duas implementações coincidem ponto a ponto (erro máximo da ordem de 10^{-15} , compatível com precisão de ponto flutuante), confirmando que a implementação via DFT é matematicamente equivalente à da biblioteca SciPy.

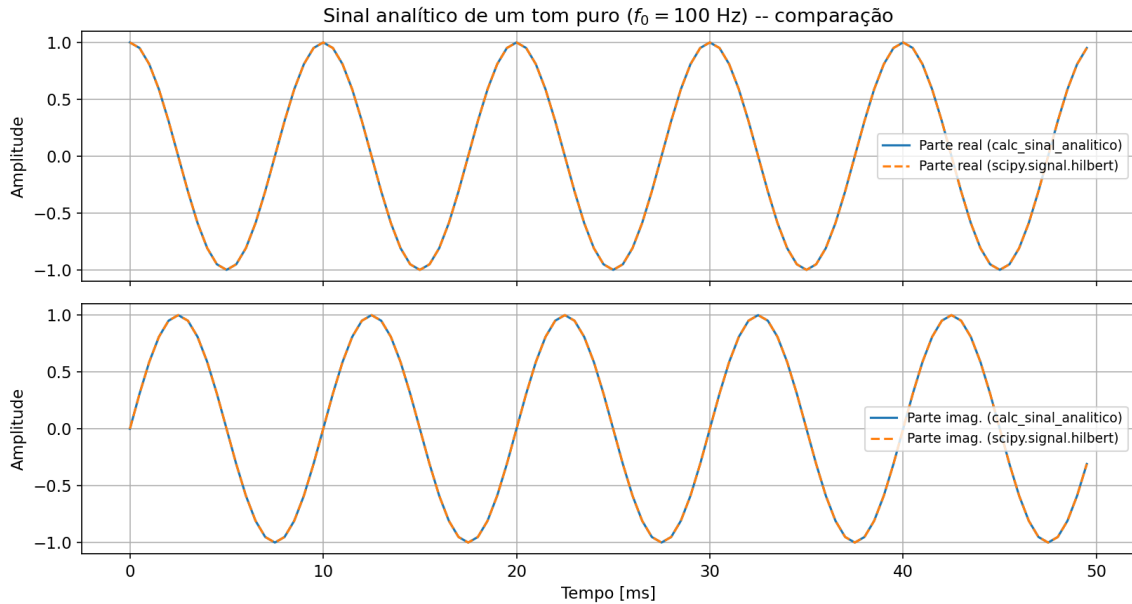


Figura 3: Partes real (topo) e imaginária (embaixo) do sinal analítico de um tom puro de 100 Hz, calculado por `calc_sinal_analitico` e por `scipy.signal.hilbert`. As curvas se sobrepõem perfeitamente.

2.3 Demodulação dos Sinais AM e FM

Tarefas 1 e 2 – Demodulação AM

O sinal analítico do sinal AM é calculado por `calc_sinal_analitico(sinal_AM)`. Sua magnitude fornece a envoltória, da qual se subtrai o nível DC unitário da portadora para recuperar o sinal modulador (Seção 1.4.1 do enunciado).

A Figura 4 mostra, no painel superior, o sinal AM (linha sólida azul) sobreposto à envoltória calculada (linha tracejada laranja), e, no painel inferior, o sinal modulador original (linha sólida azul) sobreposto ao sinal modulador recuperado (linha tracejada laranja). A concordância é excelente, e as únicas discrepâncias visíveis ocorrem nas **bordas** do sinal ($t \approx 0$ e $t \approx 5$ s), onde aparecem picos espúrios no sinal recuperado.

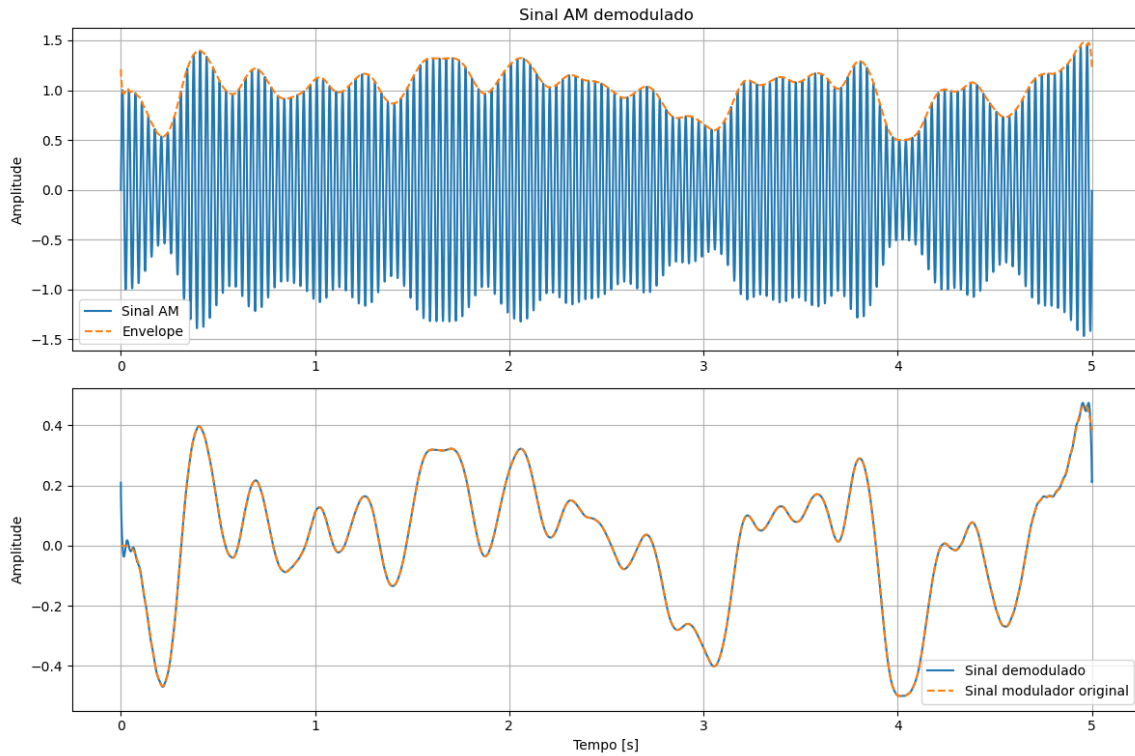


Figura 4: Demodulação AM: sinal AM e envoltória calculada (topo, zoom de 20 ms) e sinal modulador recuperado vs. original (embaixo, sinal completo).

Essas discrepâncias nas bordas ocorrem porque `calc_sinal_analitico` é calculado via DFT, que assume implicitamente que o sinal de entrada é *periódico* com período N (o comprimento do sinal). Como o sinal AM real não é periódico – e portanto, sua primeira e última amostras não se conectam suavemente –, a DFT “enxerga” uma descontinuidade artificial na fronteira circular entre o fim e o início do sinal. Essa descontinuidade se manifesta como conteúdo espectral espúrio (efeito de borda, análogo ao *leakage* espectral), que corrompe a estimativa da envoltória justamente nas amostras mais próximas das extremidades do sinal.

Tarefas 3 e 4 – Demodulação FM

De forma análoga, o sinal analítico do sinal FM é calculado por `calc_sinal_analitico(sinal_FM)`. Sua fase, desembrulhada com `numpy.unwrap`, é diferenciada numericamente para obter a frequência instantânea, da qual se recupera o sinal modulador (Seção 1.4.2 do enunciado).

A Figura 5 mostra a frequência instantânea original (linha sólida azul) sobreposta à frequência instantânea recuperada (linha tracejada laranja). Assim como no caso AM, ambos os sinais apresentam excelente concordância, e a única discrepância visível ocorre novamente nas **bordas** do sinal, pelo

mesmo motivo discutido na demodulação AM: a suposição implícita de periodicidade da DFT usada em `calc_sinal_analitico` introduz uma descontinuidade espúria na fronteira circular do sinal, degradando a estimativa da fase (e, por consequência, da frequência instantânea) nas amostras próximas às extremidades.

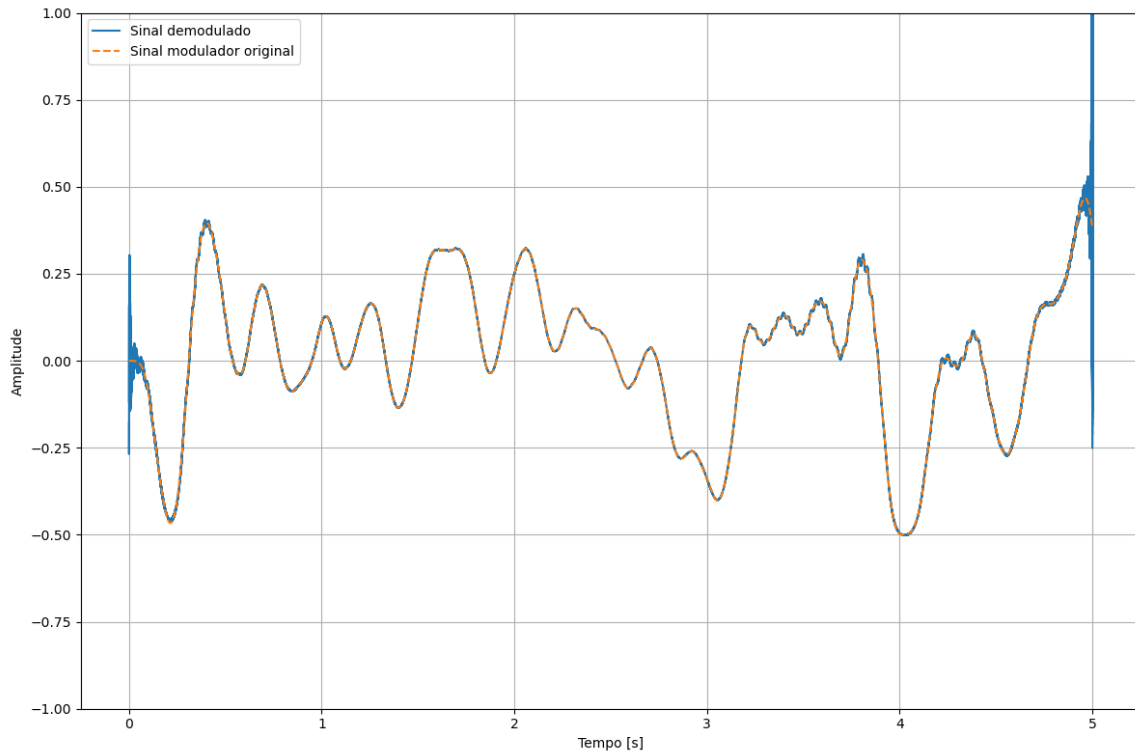


Figura 5: Demodulação FM: frequência instantânea recuperada vs. original, sinal completo (topo) e zoom longe das bordas (meio); sinal modulador recuperado vs. original, sinal completo (embaixo).

Sobre o comprimento do vetor de frequência instantânea: a frequência instantânea é obtida por diferenciação numérica (diferenças finitas) da fase desembrulhada, $f_{inst}[n] \approx (\varphi[n+1] - \varphi[n]) / (2\pi\Delta t)$. Como essa operação (`numpy.diff`) calcula a diferença entre pares de amostras *consecutivas*, um vetor de fase com N amostras produz um vetor de diferenças com apenas $N - 1$ amostras – não existe uma amostra “seguinte” à última amostra do sinal para calcular a última diferença. Por isso, o vetor `modulador_FM` (e o eixo de tempo correspondente, `t[:-1]`) possui um elemento a menos que o sinal modulador original.

Referências

1. J. Bendat, A. Piersol, *Random Data: Analysis and Measurement Procedures* (2a Edição), Wiley
2. Pacote MOSQUITO (código de referência para geração dos sinais AM/FM): <https://github.com/Eomys/MoSQITo>